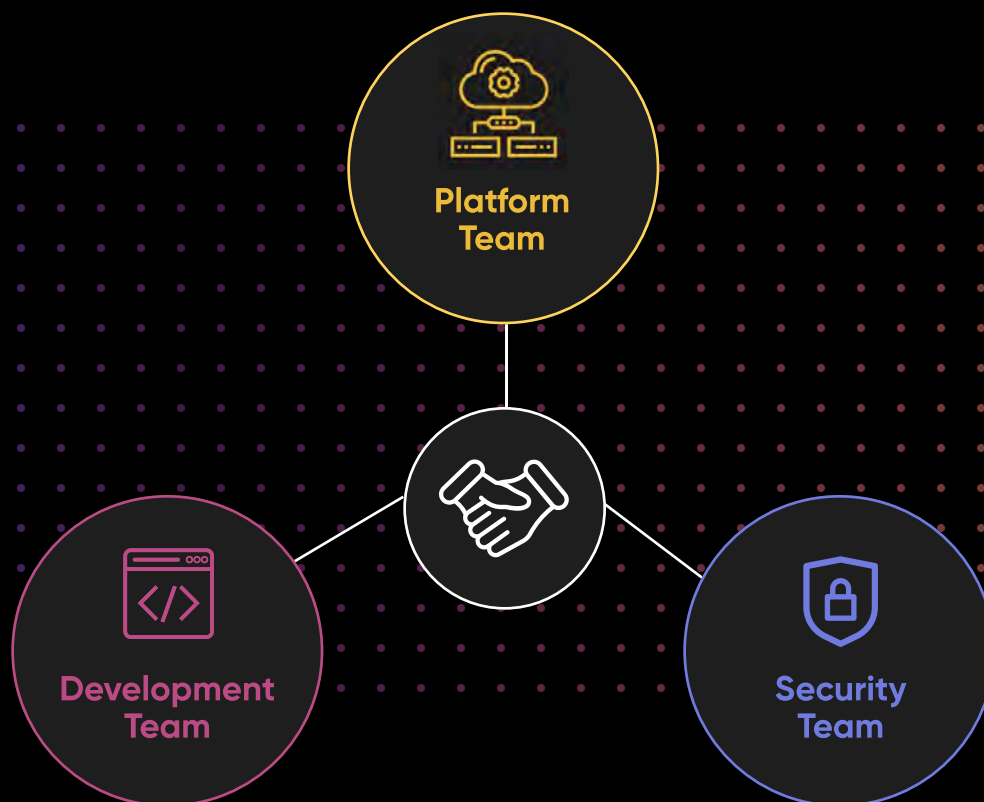


Platform Engineering:

Recommended Practices for Infrastructure as Code



Contents

Introduction

Platform Engineering:
Recommended Practices for Infrastructure as Code

Topic #1:

Code Organization and Stacks

- Setting up the scenario
- Reviewing the application
- Deploying the application with Pulumi
- First iteration of Zephyr's code

Topic #4:

Using Stack References

- Reviewing Zephyr's Status
- Linking stacks with stack references
- Zephyr's use of stack references

Topic #2:

Developer Stacks and Git Branches

- Reviewing Zephyr's Status
- First growing pains
- Stepping back to reassess
- Second iteration of Zephyr's code

Topic #5:

RBAC and Security

- Reviewing Zephyr's Status
- RBAC building blocks in Pulumi Cloud
- Assembling security for your stacks
- Inspecting Zephyr's implementation

Topic #3:

Structuring Pulumi Projects

- Reviewing Zephyr's Status
- Moving forward with Zephyr

Topic #6:

Stack Orchestration

- Reviewing Zephyr's Status
- Taking automation further
- Zephyr's Automation API Program

Conclusion

Effective infrastructure as code at scale



Platform Engineering: Recommended Practices for Infrastructure as Code

Many companies are building internal developer platforms or modern infrastructure platforms to provide developers self-service while maintaining security and compliance. The key pillars to engineering one of these platforms are developer self-service, provisioning, developer experience, security, identity, governance, and observability. A core technology that enables this is infrastructure as code (IaC) because it allows infrastructure to be created with consistency and repeatability. Pulumi IaC is adopted by many companies because it allows them to apply software engineering to their infrastructure scaling problems.

This guidebook explores some of the recommended best practices for using infrastructure as code in high scale architectures and organizations. Most of these best practices are universal to infrastructure as code in general, regardless of technology or tool. Some of these best practices are specific to Pulumi IaC because of its unique feature set. To illustrate these best practices, the guidebook uses a case study approach of a hypothetical company and platform engineering team, but very much an amalgam of current customers. This case study company, Zephyr Archaeotech Emporium, is a rapidly growing e-commerce company that has an online store. The platform team builds an infrastructure platform that is consumed by development teams building and operating the microservices of the online store.

With this context in mind, let's dive into the six topic areas of recommended practices for using infrastructure as code at scale.

Topic #1:

Code Organization and Stacks

Zephyr Archaeotech Emporium uses Pulumi to manage their online retail store. This section explores a couple common questions that users ask when working with Pulumi; specifically, where should I store my Pulumi code? And how do I support multiple environments with Pulumi? This section will provide some guidance and recommended practices around these topics, using Zephyr and their online store as the use case.

The ultimate goal of this guide is to discuss recommended practices for using Pulumi to manage a fairly complex containerized application. However, it's important to note that these recommended practices will emerge over the course of the guidebook—not all immediately, and not all in the beginning. This is a deliberate decision to allow you to see how Zephyr's use of Pulumi evolves as the company grows and their online retail store application changes to accommodate their growth.

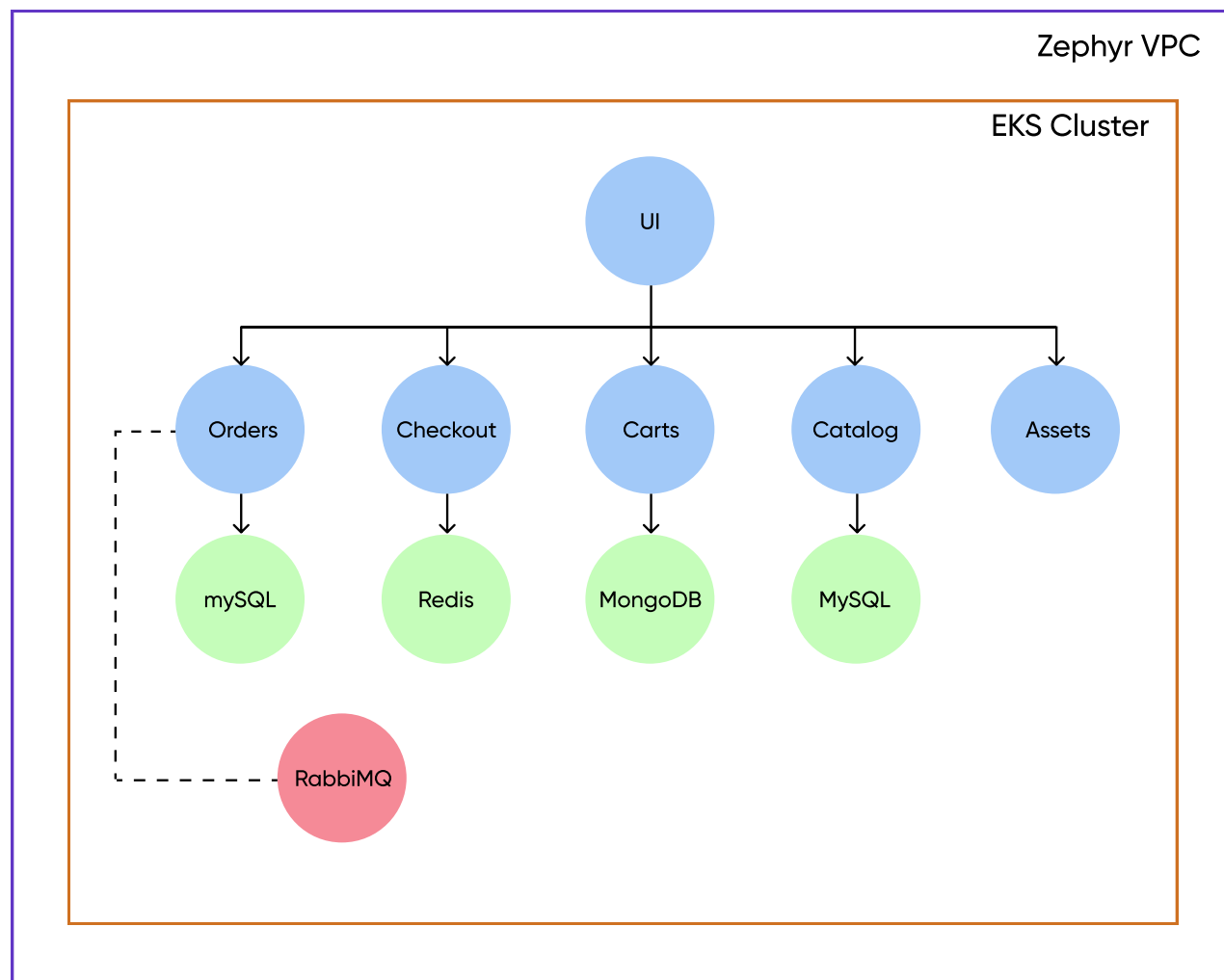
Setting up the scenario

Zephyr is short for Zephyr Archaeotech Emporium, the fictional company in our scenario. Zephyr is an online retailer that specializes in the sale of "rare arcane artifacts and replicas." Over the past few years, the company has experienced a significant increase in its online presence, making it easier for customers to purchase unique and mysterious items. Zephyr's collection includes a variety of rare and unusual objects that are difficult to find elsewhere, making it a popular destination for collectors, enthusiasts, and adventurers.

Reviewing the application

Zephyr's application is the second generation of their online store. Their first generation was a monolith that they manually deployed. It was sufficient in Zephyr's early days, but as word about Zephyr spread and the volume of traffic to their online store increased, Zephyr found it increasingly difficult to deploy small, incremental changes to the store's code. As a result, the velocity of their development efforts slowed. In preparation for the next phase of their growth—driven by robust sales of arcane artifacts—Zephyr's team evaluated a number of different architectures that would enable them to increase development velocity and give them greater flexibility in scaling different aspects of their online store. In the end, they settled on a containerized architecture deployed to Kubernetes because some of the existing team was already familiar with these technologies.

Here's a diagram of their second-generation application architecture:



As you can see in the diagram, the application has been broken down into microservices. The UI service communicates with other backend services via HTTP APIs; the various backend services communicate with various databases like MySQL and Redis for data persistence and/or caching. The backend services are responsible for implementing the functionality of a typical e-commerce site like orders, checking out, and managing the inventory catalog. While the choice of Kubernetes does afford Zephyr some much-needed functionality—like the ability to easily scale different services within the application—it also brings with it an added level of complexity that did not exist with their previous monolithic architecture. The Kubernetes manifest to deploy their application tops out at just over 1,000 lines of YAML!

Astute readers may note that Zephyr's application looks very similar to [this application](#); in truth, this guidebook uses a fork of the AWS Containers retail store example as the application in this fictional scenario. (Many thanks to the AWS Containers DA team for their hard work here!) This is an admittedly over-engineered application, but it does afford the opportunity to address a range of customer scenarios in this handbook.



Deploying the application with Pulumi

As part of the switch to their new architecture, Zephyr decided they wanted to use Pulumi to manage both the infrastructure and the application deployment. Why Pulumi? Zephyr's team recognized that moving to microservices on Kubernetes was going to make it more difficult to create reproducible, consistent implementations of their application. Zephyr's team also knew that adopting infrastructure as code would help them with fast and repeatable deployments with little additional complexity. An added bonus was being able to use programming languages they already knew.

As they prepared to embark on using Pulumi, a couple of questions came up for the Zephyr team:

- Where should they store their Pulumi code? The ability for Zephyr's developers to use a full programming language to manage the infrastructure and deployment of their online store is a huge plus, but should this code go in the same repository as their application code? Or a different repository?
- How do they address the need for multiple environments? It's clear to the Zephyr development team that they'll need multiple instances of the online store (for things like testing, QA, or development). What's the best way to handle this when using Pulumi?

Let's examine each of these issues in a bit more detail, and see how Zephyr chose to proceed.

Storing their Pulumi code

When it comes to answering the question of where to store Pulumi code relative to the application(s) it supports, there are two basic options:

1. In the same repository as your application code: This approach has the advantage of being simpler, but may not offer the necessary flexibility when dealing with multiple teams or varying levels of access control.
2. In a separate repository from your application code: Using a separate repository adds some complexity, but does provide greater flexibility to more easily address a variety of organizational requirements and varying levels of access control.

So which approach is best? That will depend on a number of different factors, many of which are outlined in the [organizing projects & stacks documentation](#). Some other factors that users need to take into consideration include:

- Who is responsible for maintaining the code? If the application code and the Pulumi code are managed by different teams, then using separate repositories may be the best approach.
- What sort of access controls are needed? If the organization needs different access controls for the application code and the Pulumi code, then separate repositories are generally needed.
- What is the relationship between the infrastructure resources managed by Pulumi and the applications? If the infrastructure is "shared," meaning it is used by multiple applications, then keeping the Pulumi code in a separate repository may be preferred. If the infrastructure is dedicated to a specific application, then keeping the application code and the Pulumi code together may be preferred.
- What about CI/CD? If an organization will use CI/CD, that may affect how you organize your code.

It's worth stressing that the answers to these questions may change over time as organizations grow and their applications evolve.

The discussion above is working from the assumption that Pulumi users will store their Pulumi code in a version control system, like Git. Strictly speaking, using a version control system isn't required for Pulumi to function (there is one exception we'll touch on later in this guide when we discuss [Pulumi Deployments](#)), but we very strongly recommend that all users use Pulumi in conjunction with version control.

In the case of Zephyr, their team felt like a monorepo approach (storing Pulumi code and application code together) was most appropriate for their specific requirements. Zephyr is a relatively small organization with a single team of developers that are responsible for managing both the cloud infrastructure as well as the development and deployment of the online store application. Zephyr's team also decided to use a single Pulumi project—for now.

Addressing the need for multiple environments

The second question Zephyr encountered is how to handle the need for multiple instances of their application. In addition to a production instance—which is the instance behind Zephyr’s public-facing online store—Zephyr also felt they needed an environment for the developers to use in testing changes to the online store.

This use case—needing to have multiple, separate instances of the infrastructure and applications created by a single Pulumi program—is exactly what Pulumi stacks were designed to address. Each stack is a separate instance of the resources created by a Pulumi program within a project. Further, each stack has its own independent state, and each stack has its own configuration values. Stacks can be short-lived (meaning the associated resources are also short-lived), or stacks can be long-lived. Aside from the cloud resources created by the Pulumi program, stacks are lightweight and simple to create or delete.

Zephyr decided to initially start with two stacks: a production stack (named “prod”) and a development stack (named “dev”). As you’ll observe throughout this guidebook, this is a decision that is easily adjusted over time as your organization’s requirements change.



Viewing the first iteration of Zephyr's code

You can view the first iteration of Zephyr's Pulumi and application code—the iteration that corresponds to the decisions described in this section—by navigating to [this GitHub repository](#). From the branch/tag dropdown, switch from the main branch to viewing the [blog/original](#) branch.

From that GitHub repository, you can also choose to deploy the Pulumi code yourself. Full instructions for deploying the code are found in the repository.

This section introduced you to Zephyr, the fictional company for this guidebook, and reviewed the application architecture for Zephyr's online store (a containerized application running on Kubernetes). Throughout the guide, this application serves as the use case around which various recommended practices are discussed.

In this section the following recommended practices were covered:

1. Use version control for your Pulumi code. Although it isn't strictly required in many cases, a number of the benefits of adopting infrastructure as code are lost without the use of version control.
2. Place your Pulumi code into version control repositories in a way that aligns with organizational requirements. There is no "right" or "wrong" approach; choose the approach that enables you to satisfy your organization's requirements around access control, code/repo ownership, and other organizational requirements.
3. Leverage Pulumi stacks to create multiple instances of cloud resources with a single Pulumi program. Pulumi stacks were created to address the need for organizations to create multiple instances of cloud resources, each with its own state and its own set of configuration data, from a single Pulumi program.

For now, Zephyr has decided to go with a monorepo approach—a single repository that contains both their application code and the Pulumi code to manage the infrastructure resources. All of the resources are defined in a single Pulumi project, with multiple stacks that correspond to development and production environments. Over the course of this guidebook, we'll see how Zephyr's use of Pulumi changes as Zephyr grows and their application evolves.

Topic #2:

Developer Stacks and Git Branches

This section explores two more questions users frequently ask when working with Pulumi in teams – namely, How can I best enable multiple developers to collaborate on a Pulumi project? And how can I use Git and Git branching to support this kind of collaboration? In this walkthrough, we'll provide some guidance and recommended practices around these topics, continuing to use Zephyr and its online store as the use case.

The ultimate goal of this guide is to discuss recommended practices for using Pulumi to manage a fairly complex containerized application. However, it's important to note that these recommended practices will emerge over the course of the handbook – not all immediately, and not all in the beginning. This is a deliberate decision to allow you to see how Zephyr's use of Pulumi evolves as the company grows and its retail application changes to accommodate its growth.

Reviewing Zephyr's Status

From the last section, The Zephyr team was off and running, managing their newly refactored online store, Zephyr Archaeotech Emporium, with a single Pulumi project and two Pulumi stacks – one for development (dev) and another for production (prod). The team had chosen to use one Git repository (a monorepo) to manage the code for the online store and its infrastructure after refactoring the store into a set of containerized microservices deployed with Kubernetes on Amazon EKS.

For a snapshot of the code as it was at the end of the first section in the guide, see the [blog/original](#) branch of the [pulumi/zephyr-app](#) repository on GitHub.

Up to now, the two-stack approach has worked well for the team. It allowed them to focus on building new features, integrating those features into the shared dev environment by developing and testing locally and merging their feature branches regularly into the base branch, main, using standard GitHub pull-request workflows. Around once a day, as the team's changes are merged into main, the team lead (and original author of the Pulumi program) deploys them into the dev environment with Pulumi with the following commands:

```
git pull origin main
cd infra
pulumi stack select zephyr/dev
pulumi up
```

This builds new container images for any services whose code has changed since the last stack update, pushes those images to [Amazon ECR](#), and updates the EKS cluster to deploy the services automatically.

Then, around once a week, as new features are ready for customers, the team lead deploys the main branch of the repository into production similarly:

```
git pull origin main
cd infra
pulumi stack select zephyr/prod
pulumi up
```

The deployment process is still a bit more manual than the team would like – but since updates typically take only a few minutes, the team lead is generally satisfied with the status quo.

The deployment process is still a bit more manual than the team would like – but since updates typically take only a few minutes, the team lead is generally satisfied with the status quo.

A few more details worth noting out about Team Zephyr's current setup:

- Both of the project's stack-configuration files, `Pulumi.dev.yaml` and `Pulumi.prod.yaml`, are checked into version control. This is safe and considered good practice (even with secrets) because it allows the team to review or change the configuration of a given stack easily by viewing or editing the code in each file.
- The main branch is the only long-lived branch in the repository; all others (e.g., developers' feature branches) are deleted once they're merged into main.
- Thanks to this workflow, the application deployed into the dev environment tracks closely with the latest code in the main branch. As such, the dev stack serves as a kind of pre-prod or integration environment that not only allows the team to dogfood new features as they're added, but also to catch bugs or other regressions before they escape into production. When things do go wrong, the team can review the list of recent stack updates to identify the source of the defect and get it fixed.
- Zephyr is using the Pulumi Service backend with a single organization (that everyone on the team belongs to) and separate AWS accounts for the dev and prod environments. Everyone on the team has read/write access to the AWS account used for development, but only the team lead has access to the AWS account used for production.

First growing pains

Life for Team Zephyr was good – for a while.

Then one day during a particularly active sprint, a developer who'd started working on an improvement to the checkout process realized it might benefit from an updated version of Redis, which appeared to be presently pinned at 6.0 in the Pulumi program:

```
// ...
const checkoutRedisDeployment = new k8s.apps.v1.Deployment("checkout-redis-deployment", {
  spec: {
    template: {
      spec: {
        containers: [{
          image: "redis:6.0-alpine",
          // ...
        ]
      }
    }
  }
})
```

A quick glance at the dev and prod stacks in the Pulumi Service confirmed this was the case.

After a quick spike to validate the enhancement locally, the developer felt confident that a Redis upgrade was the way to go – but since the deployed version of Redis would ultimately dictate the code that'd have to be written to finish the feature for real, they couldn't really be sure without testing the theory with a full deployment. What they really wanted was to be able to deploy a Redis upgrade, along with the associated code changes, into an AWS environment as much like production as possible – the dev environment, say – without having to go through the process of making a pull request, getting it merged, and waiting for the team lead to deploy the dev stack only to realize later that the change didn't actually work as expected, and be forced to revert.

In situations like these, it can be tempting to suggest that the developer simply pick up and use the dev stack – especially on small, fast-moving teams like Zephyr’s where everyone already has access to the Pulumi stack and the shared cloud environment. Indeed, Pulumi makes this sort of thing easy: to deploy their local changes into the dev environment, our developer need only check out their feature branch, select the dev stack, run `pulumi up`, and then wait a few moments for the changes to be deployed:

```
git pull origin main
cd infra
pulumi stack select zephyr/dev
pulumi up
```

This approach definitely works – but it presents at least a few problems:

1. The team would lose all of the practical benefits of having the dev stack track closely with the latest content in main.
2. The team lead’s deployment of main into dev later that day would overwrite the developer’s own feature-branch deployment. There’s a good chance the lead would notice the diff of the deployment preview and deduce that someone else on the team had possibly deployed into dev out of band:

```
kubernetes:apps/v1:Deployment (create):
  spec: {
    template: {
      spec: {
        containers: [
          [0]: {
-           image: "redis:6.0-alpine"
+           image: "redis:7.0-alpine"
            ...
          }
        ]
      }
    }
  }
  ...
```

2. But if they didn't, the developer's ad-hoc deployment would be lost. At best, it'd force both team members to drop what they were doing and get in touch to figure out what to do next. At worst, it could lead to confusion and the potential for shipping untested code.
3. In addition to raising the risk of developers overwriting each other's work, active collaboration on the same stack increases the likelihood of deployment conflicts. On the one hand, this is a good thing in that Pulumi prevents same-stack updates from running concurrently:

```
$ pulumi up  
error: [409] Conflict: Another update is currently in progress.
```

3. But on the other, it's a not-so-good thing in that it's a reduction in development velocity: The update that can't run because another update is in progress is an update that will have to wait until sometime later. The more often this happens, the longer it ultimately takes for the team to get the validation it needs to keep moving forward.

So facing these issues, what should the Zephyr team do? The dev stack had originally been created for rapid development – and that worked well, when there was only one developer working on infrastructure. With several, however, things clearly get complicated. Should the team be using a dev stack at all? If so, how? And if not, what should they be doing instead?



Stepping back to reassess

With what we've learned so far, we can return to the questions we raised at the beginning of this section and begin to sketch out a few recommendations for Team Zephyr. Those questions, again, were:

- How can I best enable multiple developers to collaborate on a Pulumi project? For Zephyr, an ideal answer to this question would give every member of the team the ability to iterate on both application and infrastructure code concurrently, deploying as often as needed without having to worry about disrupting the work or the velocity of anyone else on the team.
- How can I use Git and Git branching to support this collaboration? So far, the feature-branch/pull-request workflow has been serving the Zephyr team well. Should they continue on the same path, merging frequently to main and deploying to dev and prod from there? Or is there some alternative workflow that might serve the team better, either now or at some time in the future?

Let's answer these questions now by taking a look at how the Zephyr team handled them.

Using developer stacks

We've seen a few ways that having multiple developers share a single stack for ad-hoc deployment can lead to various problems. But we've also seen at least one situation in which being able to run ad-hoc deployments – without the risk of colliding with existing environments – can be quite valuable. In fact there are many reasons you might want to do this – and fortunately Pulumi makes it quite easy to do so.

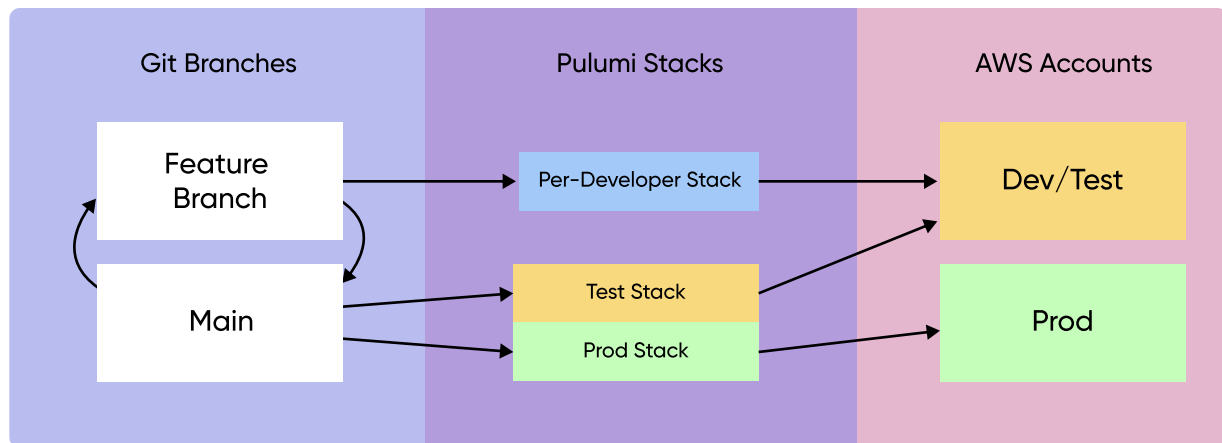
The recommended practice is to allow developers to create and manage their own Pulumi stacks. In this workflow, developers working on feature branches are able to iterate freely on both application and infrastructure, deploying as often as necessary into a shared cloud environment (typically a sandbox environment, isolated from production) and destroying cloud infrastructure when it's no longer needed.

For the developer, the typical workflow looks something like this:

1. Check out a new feature branch
2. Make changes to application and/or infrastructure code
3. Select your personal developer stack with `pulumi stack select`
4. Deploy your stack into your team's shared cloud environment with `pulumi up`
5. Commit your code and submit a pull request, optionally including a link either to the Pulumi update (e.g., when using the Pulumi Service) or to certain key resources belonging to the stack – for example, an application's primary HTTP endpoint

The benefits of a workflow like this one are many:

- No more resource-naming collisions. Because Pulumi names cloud resources uniquely by default, no two stacks will ever produce cloud resources with the same names.
- No more deployment conflicts. When every developer has a stack of their own, they can deploy to the sandboxed environment as often as necessary, without having to wait for another deployment to finish first.
- Everyone on the team has their own production-like environment to use for end-to-end testing.



For Zephyr, the quickest path to adopting this workflow was to use the dev stack as a foundation for new developer stacks. To do that, team members simply ran the following Pulumi commands:

```
pulumi stack init zephyr/developer-name --copy-config-from zephyr/dev
pulumi up
```

Developer stacks are considered ephemeral, so they're torn down with `pulumi destroy` when not in use in order to cut down on cost. Stack configuration files (e.g., `Pulumi.developer-name.yaml`) are checked into version control and stored alongside other stacks like `Pulumi.dev.yaml` and `Pulumi.prod.yaml`.

Keeping feature-branch workflows

We've also seen that the Zephyr team's existing Git workflow (short-lived feature branches merged frequently into a long-lived base branch) has worked well for them — and it turns out this workflow also works well when combined with the use of developer stacks.

For example, after creating and deploying a new stack to validate and test the proposed Redis upgrade, our Zephyr developer simply included a link to the application (deployed with their dev stack) in the associated GitHub pull request. Reviewers were then able to follow the link to the online store to see the effects of the shopping-cart change firsthand, which meant they were able to approve and get the change merged quickly and confidently knowing it'd work when deployed into production.

Naming stacks clearly

Lastly, having adopted the use of developer stacks, the Zephyr team realized the name “dev” probably didn’t work quite as well as it once had for the dev stack. Since the stack and its long-running deployment had essentially been serving (and continued to serve) as an environment for testing changes before releasing them into production, the team decided to rename the dev stack test instead. To do that, they simply ran:

```
pulumi stack select zephyr/dev  
pulumi stack rename zephyr/test
```



Viewing the second iteration of Zephyr's code

You can view the second iteration of Zephyr's Pulumi and application code – the iteration that corresponds to the decisions described in this section– by navigating to [this GitHub repository](#). From the branch/tag dropdown, switch from the main branch to the [blog/dev-stacks](#) branch.

From that GitHub repository, you can also choose to deploy the Pulumi code yourself. Full instructions for deploying the code are found in the repository.

Summarizing Topic #2

In this section, we covered several new recommended practices for working collaboratively with Pulumi, all aimed at optimizing for high team and developer velocity:

- Use short-lived feature branches and merge them frequently into a single base branch.
- Deploy the base branch regularly into a long-running pre-prod environment to continuously test the integration of the components of the system.
- Enable developers to create and use dev stacks of their own as sandboxes for development, testing, and collaboration.
- Name your stacks clearly and in alignment with the environments they deploy into – test, staging, production, etc.

Topic #3:

Structuring Pulumi Projects

In this section, you'll see how Zephyr's use of Pulumi changes as their company grows and evolves. While the ultimate goal of this guide is to discuss recommended practices for using Pulumi to manage a fairly complex containerized application, it's important to note that many recommended practices are "point in time" recommendations: best practices are context-dependent and the recommended practices for a given organization will change as the company and its infrastructure needs also change. This step-by-step guide is an example of that evolution in action: the practices that worked well for Zephyr while they were smaller and had smaller infrastructure and less teams working on that infrastructure are no longer ideal as the company has grown.

Reviewing Zephyr's Status

As a result of launching its online presence, Zephyr saw amazing success. The market for arcane artifacts and novel curiosities is booming, and Zephyr is becoming a leader in this market. Funded by this success, Zephyr has grown rapidly. Along with this growth, Zephyr's software engineering and IT teams also grew and expanded. Fortunately, Zephyr's use of per-developer stacks with short-lived ephemeral Git branches positioned the development teams well for this growth.

For a look at the state of Zephyr's Pulumi code as it stood as of the end of the second section in the guide, see the [blog/dev-stacks](#) branch of the [pulumi/zephyr-app](#) repository on GitHub.

Naturally this growth has resulted in some separation of duties. The IT team now has responsibility for the infrastructure that supports the online retail store. Within that team, some team members are focused on "core" infrastructure, while other team members are focused only on Kubernetes (a natural decision given Zephyr's use of Kubernetes in their architecture). The application team continues to remain responsible for the online store application itself.

This left Zephyr again asking some questions about the best path forward:

- Should they use a single project, or multiple projects? Zephyr had used a single Pulumi project up until now, but growth and organizational change meant they wanted to make sure they were using Pulumi in a way that best reflected their needs. What are the advantages and disadvantages of each approach?
- If multiple projects, how many projects? What's the best way to determine how many projects are needed? What factors play a role in this decision?

Structuring Pulumi projects

One project, or multiple projects? And if multiple projects, how many projects? These are the core questions the Zephyr needed to resolve. In the end, there are a number of key factors that determine the answers to these questions:

- Use case: Zephyr has only a single application (so far!), but in situations where there are multiple applications then keeping those applications (and sometimes their associated infrastructure, depending on other factors) in separate Pulumi projects is generally recommended. In some ways, "use case" is an aggregation of other factors—a new application with a distinct set of resources that do not share a lifecycle with existing resources and are potentially managed by a different team.
- Company/department/team structure: How a company/department/team is organized should certainly be taken into account when structuring your Pulumi projects. In Zephyr's early days with Pulumi, a single team managed everything so a single project was fine. Now with multiple teams, Zephyr will likely find using multiple projects to be more effective.
- Security: Closely related to the previous bullet, security is a factor that affects how projects and stacks should be structured. In Zephyr's case, the ability for one team to affect another team's resources was present when everything was a single project. With multiple projects, Zephyr can use stack permissions and role-based access control (RBAC) in Pulumi Cloud to appropriately control who can affect what.
- The relationship between resources: The relationships between resources—such as which resources are dependent upon other resources—is another factor that feeds into your Pulumi project structure. A "shared infrastructure layer" is often best handled in a separate project of its own, but may pull in resources that might otherwise have been kept with an application in its own project. (In some respects, this is a "counter-balance" to the first bullet point.) On the other hand, resources that belong exclusively to a specific application or function might be best bundled in the same Pulumi project.

- Resource lifecycle: Do resources have a shared lifecycle, meaning they are always created, updated, or deleted together? If so, then those resources should be placed in the same project. If the resources do have a shared lifecycle, then placing them in the same project simplifies lifecycle management. A great example of resources with a shared lifecycle would be a Kubernetes Deployment and the associated Kubernetes Service.
- Change rate of resources: Some resources may change very infrequently. Consider the example of an AWS VPC and associated objects. Once it is created, it may not change for extended periods of time, if ever. On the other hand, some resources may change frequently (in other words, these resources have a high change rate). A fast-moving software development team may revise the Kubernetes Deployment regularly to pull in a new container image with updated code. In a single project, combining resources that have a high change rate with resources that have a low change rate may be suboptimal.
- Blast radius: Finally, there is the consideration of what happens in the event of human error, and limiting the “blast radius.” With a single project, all the resources in that project are within the blast radius—all of them could potentially be affected in the event of human error. Using separate Pulumi projects can limit the blast radius. It’s not something that should drive the decision all by itself, but it is a factor to be considered.

Now that you have an idea of some of the factors that affect how users structure their Pulumi projects, what are the advantages and disadvantages of each approach? For single projects, the key advantage is simplicity. There’s no denying that a single project is simpler and less complex than working with multiple projects. This advantage, though, is also the key disadvantage. A single project does not afford the flexibility to let different teams manage different aspects of the infrastructure, for example, nor does it offer the ability to split resources that don’t have a shared lifecycle. And, as mentioned earlier, blast radius is a concern.

For multiple projects, the key advantage is flexibility. It’s true that the added flexibility comes at the cost of some added complexity. However, you gain the ability to separate infrastructure resources according to team ownership/responsibility, relationship to other resources (i.e., grouping resources in a “shared infrastructure layer”), or any of the other dimensions mentioned above.

In some ways, the decision points for single vs. multiple projects are analogous to the decision points for monolith vs. microservice: independent scaling and flexibility versus simplicity.

In general, most customers end up using multiple Pulumi projects per use case. As you’ve seen, though, there are a number of factors that influence that decision and will affect how many projects end up in use.

Moving forward with Zephyr

Zephyr had a number of factors in play, not the least of which was the organizational change resulting from their growth and success. In the end, Zephyr decided to restructure their use of Pulumi for the online retail store into three separate projects:

- An infrastructure project: This project handled the AWS VPC and related infrastructure resources.
- A platform project: This project managed the Kubernetes cluster(s).
- An application project: This project handled the actual online retail store itself.

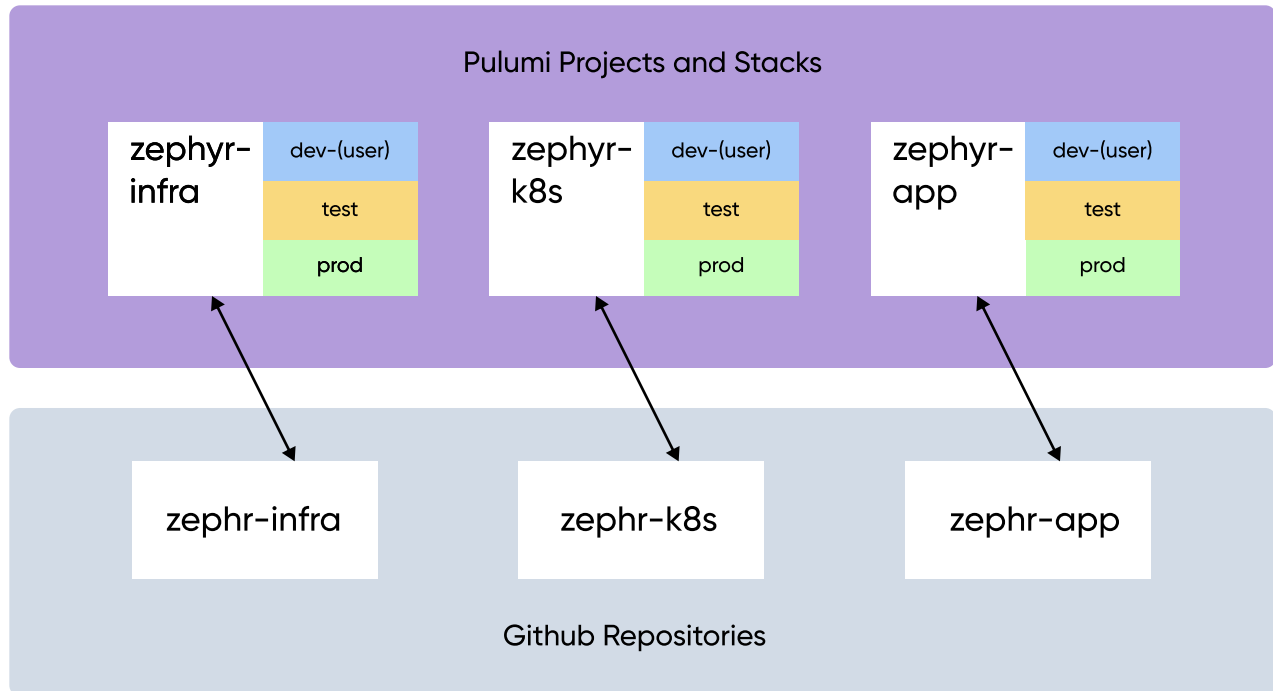
You can use a combination of code refactoring and `pulumi import` to assist in the process of restructuring Pulumi projects, but in some cases it may be easier to create new infrastructure and migrate over to it.

Earlier you read about some of the options for where to store Pulumi code relative to the applications it supports. At that time, Zephyr had decided to go with what's called a monorepo approach, meaning that the Pulumi code was stored in the same repository as the application code. In conjunction with the project reorganization, Zephyr also decided to change how their code was organized:

- Code for the infrastructure project moved to a separate "zephyr-infra" repository
- A new "zephyr-k8s" repository housed the code for the platform project
- The code for the online retail store application itself remained in the same "zephyr-app" repository

Note that relocating code (to a different GitHub repository or different filesystem location within a repository) generally has no impact on Pulumi. As long as the project name and stack name do not change, then changing the filesystem location won't affect anything—keeping in mind, of course, that filesystem paths and such may need to be adjusted in code to account for any such changes.

After restructuring their projects, Zephyr's arrangement of Pulumi projects and stacks, along with their associated GitHub repositories, looks something like this:



These repositories are available for you to review on GitHub ([zephyr-infra](#), [zephyr-k8s](#), and [zephyr-app](#)); use the branch/tag selector to find the blog/multi-project branch in each repository to see the state of the code as of this section. Be aware that the process for standing up the Zephyr online store is now a bit more complex; check the instructions in each repository for more details if you'd like to give these projects a spin.

Summarizing Recommended Practice #3

This section covered the following recommended practices for working with Pulumi:

- Use multiple Pulumi projects when you need the added flexibility that multiple Pulumi projects offer.
- Align your Pulumi projects according to external factors like company/department/team structure, resource dependencies, lifecycle, and use case/application.
- Change the filesystem location for Pulumi code as needed without impacting the resources managed by that code, as long as the Pulumi project name and stack name don't change.

Topic #4:

Using Stack References

This section is a complement to the earlier references on structuring Pulumi projects, discussing how Zephyr uses Stack References to link their projects together and sharing some recommended practices around the use of Stack References.

As you may have read in earlier Zephyr guides, the ultimate goal of the Zephyr use case is to share recommended practices on the use of Pulumi to manage your infrastructure and application resources (using a fictional company and a somewhat complex containerized application as the use case). The compendium exposes those recommended practices over time—not all right away, and not without also discussing the context for the recommendations. This is deliberate, demonstrating how “point-in-time” recommendations change based on the needs of the company and its requirements.



Reviewing Zephyr's Status

Zephyr's infrastructure footprint and Pulumi codebase is the same; that is, Zephyr now has three Pulumi projects (zephyr-infra, zephyr-k8s, and zephyr-app). Each of these projects handles a different aspect of Zephyr's online store:

- The "zephyr-infra" project handles base infrastructure (VPC and related constructs).
- The "zephyr-k8s" project handles the Kubernetes platform layer, building atop the infrastructure components provided by the "zephyr-infra" project.
- The "zephyr-app" project handles the deployment of the online store services onto Kubernetes, layering on top of the platform provided by the "zephyr-k8s" project.

For Zephyr, it came down to supporting multiple teams and allowing the application to evolve independently of the infrastructure that supports it.

You can continue to use the [blog/multi-project](#) branch in the GitHub repositories to see the state of Zephyr's code and projects for this section.

It's clear that there are dependencies across projects: The zephyr-k8s project needs to know the VPC ID and the subnet IDs from the zephyr-infra project. Similarly, the zephyr-app project needs to have access to the Kubernetes cluster details from the zephyr-k8s project. As the Zephyr team went about building the code that's now in use in their multi-project architecture, they needed to decide how best to handle these cross-project dependencies. The Zephyr team knew that hard-coding output values from one project as configuration values in another stack wasn't the ideal way (and in fact Pulumi strongly recommends against hard-coding output values), but what was the best solution to use? The answer to that question is stack references.

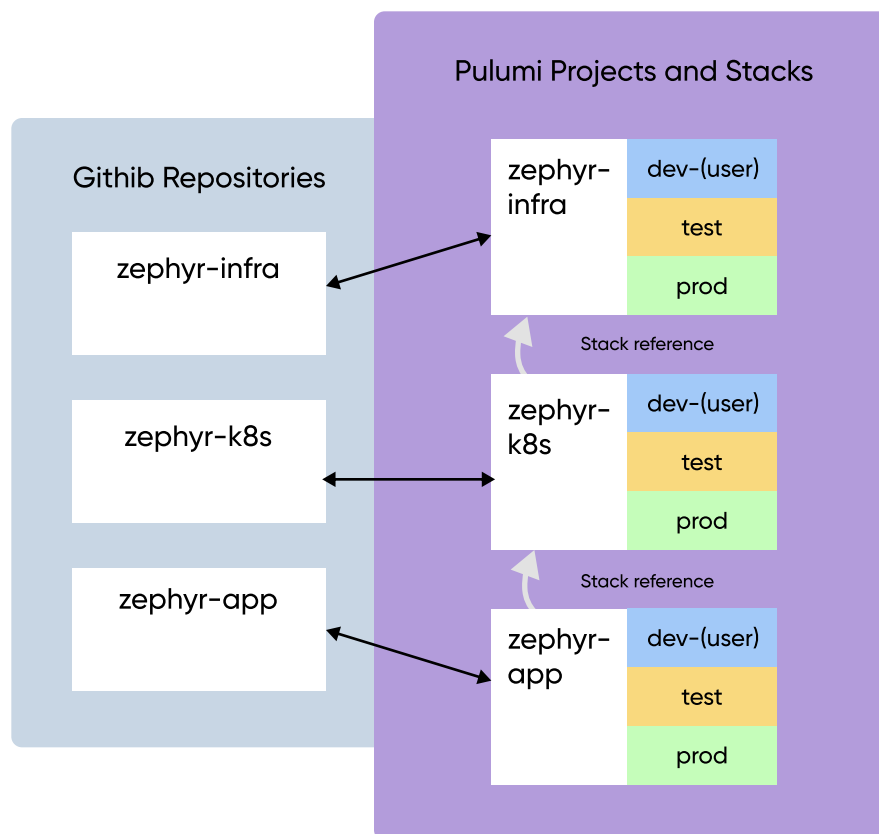
Linking stacks with stack references

Stack references allow you to access output values from one stack in another stack. In other words, stack references offer a way to programmatically “link” stacks in different projects for the purpose of sharing information. You can find more details [in the documentation on stack references](#).

Using stack references, the Zephyr team can make the necessary connections they need:

- The code handling the Kubernetes cluster can reference the VPC ID and subnet IDs from the base infrastructure layer.
- Similarly, the code that is responsible for deploying the online store services can reference the necessary Kubernetes connection details (the Kubeconfig) for the Kubernetes cluster.

You can see how the Zephyr team did this for their projects, including code samples, toward the end of this section. This graphical representation also illustrates Zephyr’s current repository and project structure as well as the stack references:



It's worthwhile to note that any information that needs to be accessible from another stack via a stack reference must be exported as a stack output in the source stack. If you don't mark it as a stack output, then it can't be used in a stack reference. Adding stack outputs after the fact requires little effort and has no impact on existing infrastructure.

Also, any value retrieved via a stack reference is treated as a Pulumi Output, and therefore may require some extra work to transform values (such as the use of `Output.apply`). The recent addition of `OutputDetails` support in Pulumi—you can read more about `OutputDetails` in this blog post announcing the functionality—helps considerably in this situation. Some SDKs also have language-specific mechanisms that can help; for example, using Go's `.AsStringArrayOutput()` method on a `StackReference.GetOutput` statement makes referencing subnet IDs from other project much easier.

While stack references are conceptually straightforward and not difficult to implement, there are some recommended practices to be mindful of regarding the use of stack outputs and stack references:

1. When it comes to stack outputs, export what is needed. If it needs to be accessed from outside the stack, export it; otherwise, don't. Why? Stack outputs form the application programming interface (API) by which other Pulumi programs interact with this Pulumi program. Like in any API implementation, expose what the user cares about or is likely to need, and leave the rest as an implementation detail. You can add more stack outputs after the fact with only minimal code changes, and as has been mentioned already this is done with a quick `pulumi up` that has no effect on existing infrastructure (you're modifying the stack object itself).
2. Be judicious in your use of stack references. This is sort of the "flip side" of the previous bullet. If you need to reference an output from another stack, then do so; otherwise, don't. If you find yourself needing lots of stack references, then the way in which you've structured your projects may need to be re-examined so as to minimize the coupling between projects. Additionally, there is a potential "speed of light" concern here; if where you are executing your Pulumi code is across a high-latency link from where you store your Pulumi state, stack references will be affected by that latency.
3. If you need to expose lots of outputs on a stack, consider exporting a structured data object. It's possible to construct a JSON object (or dict or struct, depending on your language) to hold all the stack outputs, and then export that object. Then a stack reference can read that object, resulting in a single call to the backend. Be aware, though, that you'll need to write the necessary code to understand/import/unmarshall that JSON object in the referring stack, so there is a small amount of extra work required in this situation.

4. In line with parameterizing as much of your code as possible, also be sure to parameterize your stack references. A stack reference is built using an organization name, a project name, and a stack name. Don't hardcode these values; instead, use configuration values to allow the users to specify from which source stack(s) the values will be referenced. You'll see an example of that in the next section.
5. Plan for handling cascading stack updates. If a stack has outputs being consumed via stack references, what happens when those stack outputs change? The Pulumi Service supports webhooks that can help, and Pulumi is currently exploring options for having Pulumi Deployments help with this (see this GitHub issue). In practice this should be uncommon, and frequent occurrences may be a good indicator that your resource partitioning (how resources are split among multiple projects) needs to be adjusted.

You'll want to keep the security of your stacks in mind when planning for the use of stack references. The teams managing the stacks where the stack references exist will need at least read-only access to the stacks where the outputs exist.

Examining Zephyr's use of stack references

With these recommended practices in mind, you can examine the Zephyr team's implementation to see how they put these recommendations into action.

- In the code for the base infrastructure stack, you can see that Zephyr exported the essential values needed by the Kubernetes platform stack:

```
// Export some values for use elsewhere
export const vpcId = eksVpc.vpcId;
export const privSubnetIds = eksVpc.privateSubnetIds;
export const pubSubnetIds = eksVpc.publicSubnetIds;
```

- In the Kubernetes platform stack, the Zephyr team parameterized the values needed for the stack reference. This particular approach, by the way, is key to preserving the per-developer stacks to which the Zephyr team has grown accustomed (each developer needs to specify the correct organization, project, and stack name):

```
// Grab some configuration values
// Some code omitted here
const infraOrgName = config.require("infraOrgName");
const infraProjName = config.require("infraProjName");
const infraStackName = config.require("infraStackName");

// Create a StackReference to get information from base stack
const infraSr = new pulumi.StackReference(`${infraOrgName}/${infraProjName}/${infraStackName}`);
```

- As with the base infrastructure stack, only the key value needed by other stacks is exported (in this case, the Kubeconfig required to access this cluster):

```
// Export some values for use elsewhere
export const kubeconfig = eksCluster.kubeconfig;
```

- Finally, in the application stack, the stack reference values are again parameterized, and the Kubeconfig—necessary for the application stack to deploy onto the provisioned Kubernetes cluster—is referenced via a stack reference:

```
// Grab some configuration values
const config = new pulumi.Config();
const k8sOrgName = config.require("k8sOrgName");
const k8sProjName = config.require("k8sProjName");
const k8sStackName = config.require("k8sStackName");

// Create a StackReference to get Kubeconfig from base stack
const kubeSr = new pulumi.StackReference(`${k8sOrgName}/${k8sProjName}/${k8sStackName}`);
```

All of the GitHub links in the paragraphs above reference the blog/multi-project branch in each repository, which has the code as of this section.

Summarizing Topic #4

This section covered the following recommended practices for working with Pulumi:

- Export the minimum of what's needed from your stacks. It's not difficult to add stack outputs later, and you can do it with no impact to existing infrastructure.
- Be judicious in the use of stack references. Reducing stack references where possible helps keep projects from becoming overly coupled together, much in the same way as minimizing stack outputs.
- Use a structured data object for large numbers of values that need to be referenced. This enables Pulumi to retrieve all the data with a single call, but be aware you'll need to write extra code to understand the data structure in the referring stack.
- Parameterize your stack references. Don't hardcode organization, project, or stack name values. Instead, pass these in as configuration values (with default values applied, where applicable).
- Plan to build mechanisms to handle cascading stack updates. Using the Pulumi Service's webhook functionality can be useful here. Frequent instances of cascading updates might be an indicator that your project structure needs to be revisited for optimal resource partitioning.



Topic #5:

Role-Based Access Control and Security

This section continues our guide focused on IaC recommended practices. In earlier sections, we introduced Zephyr Archaeotech Emporium, the fictional company that sits at the center of this guide, and discussed Zephyr's primary use case for Pulumi: managing their online retail store. You read how Zephyr's initial use of Pulumi changed to incorporate the use of short-lived per-developer stacks. Later, as Zephyr continued to grow, you saw how Zephyr restructured their Pulumi projects and stacks, and incorporated the use of Stack References. This exploration is a complement to the section on structuring Pulumi projects, concentrating on the use of role-based access control (RBAC) and security in Zephyr's multi-project configuration.

The ultimate goal of this guidebook is to discuss recommended practices for using Pulumi to manage a fairly complex containerized application. You've seen these recommended practices emerge over the course of the guide—not all immediately, and not all right away. Structuring the guidebook in this way is a deliberate decision; many recommended practices are “point in time” recommendations: they are context-dependent and the recommendations for any given company, like Zephyr, may change as the company and its needs change.

Reviewing Zephyr's Status

So far, you've read about how Zephyr started out with a single project and a single Git repository. Adding short-lived per-developer stacks didn't really impact this structure. However, as the company grew, Zephyr's project and Git repository structure evolved into its current form:

- The "zephyr-infra" project handles base infrastructure (VPC and related constructs).
- The "zephyr-k8s" project automates the Kubernetes platform layer, building atop the base infrastructure components supplied by the "zephyr-infra" project.
- Finally, the "zephyr-app" project handles the deployment of the online retail store application onto Kubernetes, layering on top of the platform provided by the "zephyr-k8s" project.

One of the reasons for this structure is what we covered earlier: "security is a factor that affects how projects and stacks should be structured. In Zephyr's case, the ability for one team to affect another team's resources was present when everything was a single project. With multiple projects, Zephyr can use stack permissions and role-based access control (RBAC) in Pulumi Cloud to appropriately control who can affect what."

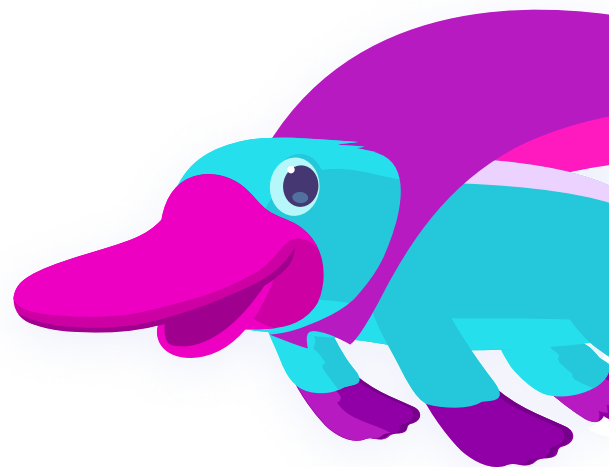
The last part of that quote—the part about being able to "appropriately control who can affect what"—is another way of describing what's known as the principle of least privilege. If you're unfamiliar with this term, we recommend following the link and getting more information, because the principle of least privilege stands at the core of all the recommended practices you'll read in this section.

Pulumi supports a number of self-managed backends, including Amazon S3 (and S3-compatible servers like Minio or Ceph), Azure Blob Storage, and Google Cloud Storage. The RBAC functionality described in this section applies only to the managed Pulumi Cloud backend.

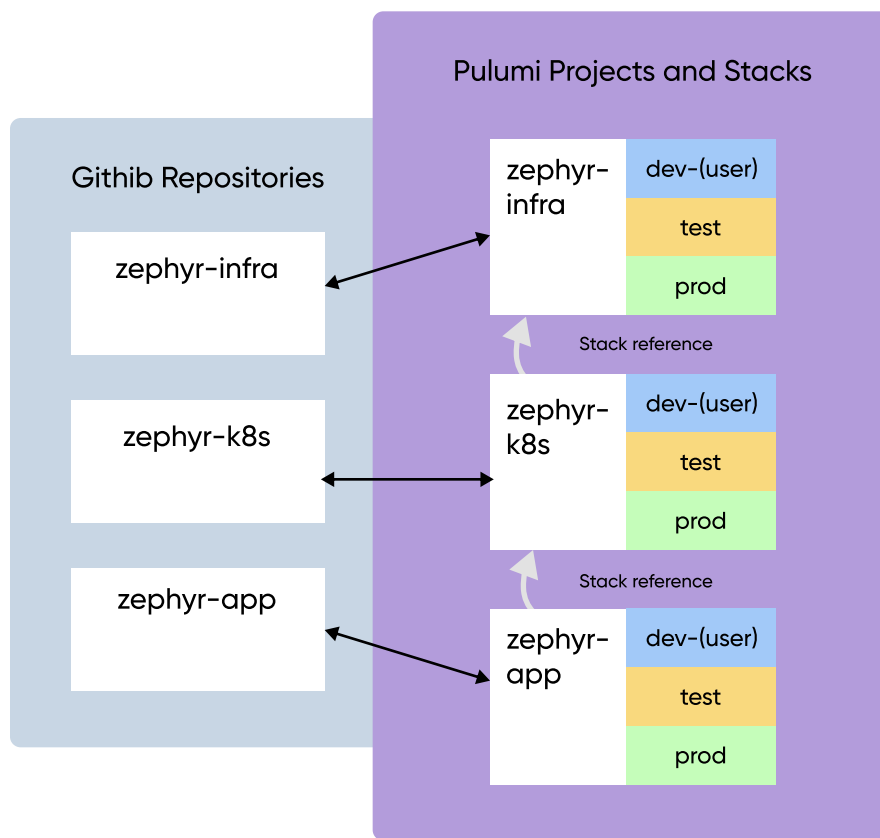
Understanding the RBAC building blocks in Pulumi Cloud

To understand how to secure your Pulumi projects and stacks in Pulumi Cloud, it is important to first explain the basic building blocks you'll use:

- **Organization:** An organization represents a customer/company within Pulumi Cloud. Generally speaking, a single company like Zephyr is represented by a single organization.
- **Member:** Each member represents a user account that is part of an organization. In Zephyr's case, each Zephyr employee that needs to interact with Pulumi is a member of Zephyr's organization in Pulumi Cloud.
- **Team:** A team is a grouping of members, and is the primary component involved in assigning per-stack permissions in Pulumi Cloud's RBAC model. How you structure teams within your organization will depend on a great many different factors we'll explore in this section.
- **Stack:** A stack is a separate instance of the resources created by a Pulumi program within a project. Each stack has its own independent state and its own configuration values. With respect to RBAC, each stack is a separate entity to which one or more teams can be granted permission. In Zephyr's case, there is a long-lived production stack for each of the separate projects, and a persistent (or long-lived) test stack for each of the separate projects. There are also the per-developer stacks, which are typically short-lived.



This diagram shows the stacks and projects for Zephyr's online store:



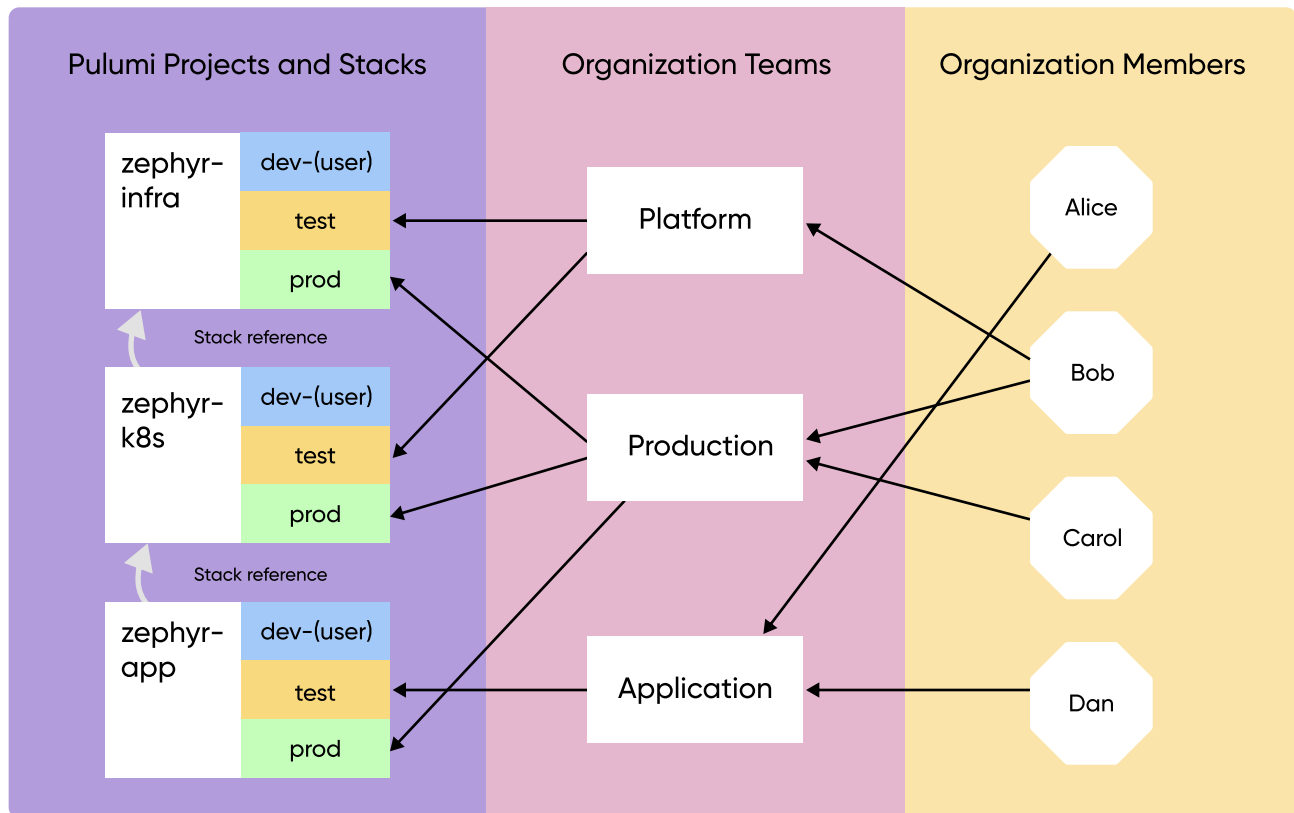
A couple of important points emerge from the above diagram:

1. The relationship between stacks and teams is many-to-many. That is, any given stack can have many teams assigned permissions, and any given team may be assigned permissions to many stacks.
2. Similarly, the relationship between teams and members is also many-to-many (any given team can—and almost always will—have multiple members, and any given member can be part of multiple teams at the same time).

Assembling security for your stacks

The first, and perhaps the most important, recommended practice with regards to securing your stacks is to actually use Pulumi Cloud's RBAC functionality. While this may seem obvious, there are practical implications to this statement that are useful to explore.

Pulumi Cloud has the ability to set a default stack permission for all members of an organization. In the screenshot below, you can see that the default permission has been set to Write:



In this configuration, you are effectively disabling Pulumi Cloud's RBAC functionality. Permissions in Pulumi Cloud are additive; that is, permissions only state what you can do, not what you can't do. Therefore, if the default stack permission is set to Write, then assigning a team read-only permissions to a stack has no effect—because members of that team can read from the stack (by virtue of their inclusion in a team) and can write to the stack (by virtue of the default stack permission). The team members' read-only permission is added to the default stack permission for their effective permission.

For this reason, Pulumi recommends setting the default stack permission to Read or None. Setting it to Read means that all members of the organization can see and read from stacks that aren't their own, but can't change or modify the others stacks. Setting it to None means that members have permissions to their own stacks by default—gaining any form of access to stacks other than their own requires being added to a team that has permissions to those stacks.

When it comes to assigning a team access to a stack, there are three options:

- Stack reader (read-only access; also called "Stack reader access" in some areas of the Pulumi Cloud UI)
- Stack editor (read/write access, but can't delete the stack; also called "Stack write access" in the Pulumi Cloud UI)
- Stack admin (full access; you may also see it called "Stack admin access")

When the "Allow organization members to create stacks and transfer stacks to this organization" is checked, like in the screenshot above, then the user (organization member) that creates a stack automatically becomes the stack admin for that stack. The user's ability to delete the stack, however, is affected by the "Allow stack admins to delete stacks" setting.

Full details on what each of these permissions grants is detailed in [the projects and stacks documentation](#) (see the "Stack Permissions" section). In accordance with the principle of least privilege, Pulumi recommends you grant the minimum level of access necessary in order for a user (organization member) to meet their job responsibilities. You accomplish this by adding the member to a team, and then granting the team the minimum level of necessary access. You can't assign stack permissions directly to users.

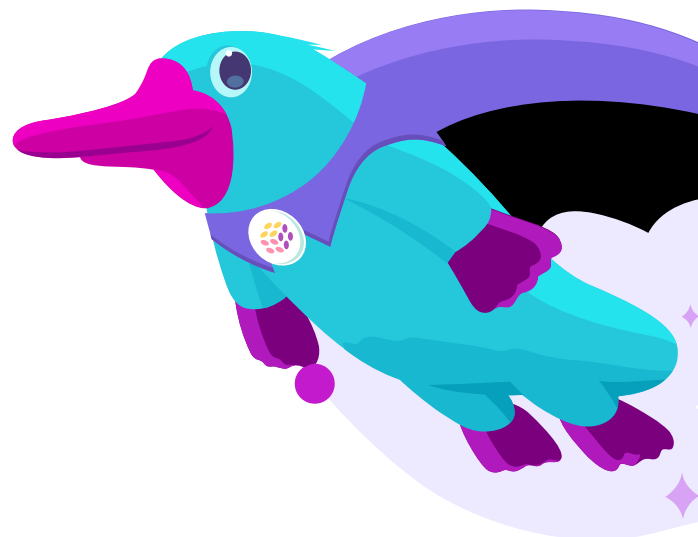
Because team membership is central to the use of stack permissions, Pulumi has several ways to make it easier to manage the relationship between teams and stacks:

1. You can use the Pulumi Cloud user interface to assign a team access to a stack.
2. You can use Pulumi Cloud's REST API to control team access to stacks.
3. You can use the Pulumi Cloud provider for Pulumi to programmatically manage team access to a stack using Pulumi itself.
4. You can use the `--teams` flag to `pulumi stack init` to assign stack editor access to one or more teams when you create the stack.

In general, Pulumi recommends declaratively managing your Pulumi Cloud RBAC configuration with the Pulumi Cloud provider (formerly known as the Pulumi Service provider), unless you have specific requirements that would prevent it. However, be aware that the Pulumi Cloud provider doesn't allow you to change a member's role in an organization (every member is either a "member" or an "admin" within an organization); that would have to be done via the Pulumi Cloud user interface or REST API.

Pulumi also recommends using the `--teams` flag with `pulumi stack init` to ensure that new stacks are properly assigned team permissions upon creation. You can then go back and fine-tune those team permissions—which are set to "Stack editor" or "Stack write access" when you use the `--teams` flag—using a Pulumi program with the Pulumi Cloud provider. Alternately, teams can use a team access token when creating a new stack, and that will also grant the team permission to the newly-created stack.

Finally, Pulumi recommends allowing users to create their own stacks. Otherwise, only an organization admin would be allowed to create new stacks, and this is likely to cause a bottleneck in provisioning cloud infrastructure—one of the very issues that infrastructure as code tools like Pulumi are attempting to solve! (It may be necessary in some cases to restrict this functionality, such as in certain highly regulated industries or verticals.) Preventing users from creating their own stacks would also prevent the use of per-developer stacks like Zephyr uses, which as you've seen can be enormously helpful in accelerating development and deployment velocity.



Summarizing Topic #5

The section covered the following recommended practices for securing your stacks in Pulumi Cloud:

- Use Pulumi Cloud's RBAC functionality by setting your organization's default stack permissions to Read or None. This is generally the safest way to implement the principle of least privilege, unless you have a good use case for allowing all organization members full write access to all stacks.
- When assigning team permissions, grant the minimum level of access necessary. If a user isn't responsible for making updates to a stack, then give that user stack reader permission through a team membership (remember that you can't assign stack permissions directly to organization members, you have to use a team). Keep in mind, though: if the organization level stack permission has been set to Read, then members already have the equivalent of stack reader permissions on all stacks, so creating a team to grant stack reader permission is unnecessary and superfluous.
- Declaratively manage your Pulumi Cloud RBAC configuration where possible using the Pulumi Cloud provider from within a Pulumi program.
- Use the `--teams` flag with `pulumi stack init` to ensure that teams have stack permissions upon stack creation. You can also use a team access token to accomplish the same result.
- Allow users to create their own stacks. Generally, turning this functionality off (found in the Settings > Access Management screen of Pulumi Cloud) will end up creating a bottleneck, as only organization admins would be allowed to create new stacks.

Topic #6:

Stack Orchestration

So far in this guide, you've seen how Zephyr Archaeotech Emporium—the fictional company at the center of this guide—uses Pulumi to manage their online retail store. You read how Zephyr's initial use of Pulumi changed to use short-lived per-developer stacks. Later, as Zephyr continued to grow, you saw how Zephyr restructured their Pulumi projects and stacks, incorporated Stack References, and used Pulumi Cloud's role-based access control (RBAC) functionality to control access to their stacks. This part focuses on how Zephyr takes advantage of the [Pulumi Automation API](#) to bring an even greater level of orchestration to the stacks that represent their online store.

The ultimate goal of this guidebook is to discuss recommended practices for using Pulumi to manage a fairly complex containerized application. These practices have unfolded organically, as a direct response to Zephyr's evolving needs. The aim is to demonstrate that recommended practices (or best practices) are not set in stone, but are rather “point in time” recommendations that adapt as your company grows.

Reviewing Zephyr's Status

Over the course of this guide, you've seen how Zephyr's use of Pulumi has grown and changed. For the sake of completeness, here's a quick recap:

- Zephyr started with a single project in a single repository with two stacks, one for production and one for testing.
- To unlock developer velocity, Zephyr quickly added per-developer stacks.
- As Zephyr grew and expanded, their single Pulumi project grew into three different Pulumi projects: one for base infrastructure, one for their Kubernetes platform, and one for the online store application.
- Accompanying the switch to multiple projects, Zephyr implemented stack references (to pass information between stacks in different projects) and applied role-based access control (RBAC) to the stacks in Pulumi Cloud.

Since we last checked in on Zephyr, one significant change has emerged. As traffic to the Zephyr online store continued to increase, Zephyr's operations team was finding it difficult to scale the relational databases running on Kubernetes (specifically, the orders and catalog databases). After evaluating their options, the team decided to migrate to [Amazon Aurora](#) with MySQL compatibility. Using the considerations outlined for structuring Pulumi projects, they decided to create a fourth Pulumi project, named "zephyr-data", to manage the data infrastructure for the online store. Initially, this data infrastructure project manages only the Aurora clusters and database instances for the orders and catalog databases.

Because Zephyr was already using a modular approach to their overall application stack, inserting a new project for managing the online store's data layer involved only minimal changes:

- A new Pulumi program had to be written. This new program used stack references to leverage the existing infrastructure created by the base infrastructure stack and allowed traffic to the databases from the security group created by the Kubernetes platform stack. You'll find this new program in [the zephyr-data repository](#) on GitHub (look in the blog/automation-api branch).
- The Pulumi program for the Kubernetes platform stack only needed to export the security group created for the cluster nodes.
- The Pulumi program for deploying the online store microservices referenced the database endpoints created by the new data infrastructure stack, and no longer spun up containerized instances of MariaDB for the orders and catalog databases.

The resulting four project architecture looks like this:

Permissions

Stack permissions

Stack permissions for all members

WRITE ▼

☒ Allow organization members to create stacks and transfers stacks to this organization

☐ Allow stack admins to transfer stacks to another organization

☒ Allow stack admins to delete stacks

Team permissions

☐ Allow organization members to create teams

Save permissions

Using a modular approach with multiple projects has its benefits. However, one of the trade-offs of splitting your IaC code into separate Pulumi projects is some additional relative complexity. With additional complexity comes additional room for error. To stand up a production-like environment, developers now had to stand up four different stacks (base infrastructure, Kubernetes platform, data infrastructure, and online store) in a specific order. If a user gets the order wrong, then the online store won't come up properly.

With only four projects, this is not an insurmountable challenge by any stretch. However, what if it was 10 projects? Or 15 projects? Those numbers are not out of reach! Right now Zephyr has a single project to deploy all the various services involved for the online store. What if the application team decides to split up that project so that individual services can be deployed independently of each other? (After all, being able to deploy microservices independently of one another is one of the benefits of such an architecture.)

The Zephyr team started asking some questions: How can we make this process simpler? How can we remove room for error? How can we prepare our teams for greater scale?

Taking automation further

The challenges facing the Zephyr teams are not unique to Zephyr. Many IaC vendors recommend using a modular approach. Unfortunately, most IaC products rely on less-than-ideal approaches—like Bash scripts or Makefiles—to “stitch” together the different modules.

In some cases, organizations can use existing build automation tools and CI/CD pipelines to assist. What's really needed, though, is a way to programmatically control the IaC tooling, and that's what Pulumi's Automation API is. The Pulumi Automation API allows Zephyr's teams to use a general purpose programming language to automate Pulumi operations like performing an update, refreshing the stack state, or destroying a stack. Like Pulumi itself, the Pulumi Automation API supports multiple languages and supports multiple state backends, including Pulumi Cloud. Further, there is no requirement that Pulumi Automation API programs are written in the same language as the Pulumi programs they automate. In the case of Zephyr, which thus far has chosen to use TypeScript for their Pulumi programs, this means they are free to use any supported language for their Pulumi Automation API programs.

When the Zephyr team started evaluating options for simplifying the deployment of their online store and associated infrastructure, they decided to move forward with the Pulumi Automation API for these reasons:

- Tight integration with Pulumi and Pulumi Cloud
- Support for Pulumi Deployments
- Support for a broad range of programming languages, just like Pulumi itself
- No requirement to use the same language as the Pulumi programs being automated (in fact, a Pulumi Automation API program could automate multiple programs written in different supported languages!)

Let's take a look at Zephyr's initial foray into using Pulumi Automation API.

Digging into Zephyr's Automation API Program

You can view the Pulumi Automation API program that Zephyr wrote in the local-source directory of [the pulumi/zephyr-automation repository](#) on GitHub, in the `blog/automation-api` branch.

Zephyr's team decided to write their Pulumi Automation API program in Golang. One of the first things you'll note about the Pulumi Automation API program is that it looks more like a "regular" Go program than a typical Pulumi Go program.

For example, a typical Pulumi Automation API program written in Go starts out like any other Go program:

```
func main() {  
    // All your Go code is here  
}
```

In a typical Pulumi Go program, on the other hand, the `main()` function starts off with a `pulumi.Run` call, like this:

```
func main() {  
    pulumi.Run(func(ctx *pulumi.Context) error {  
        // All the Pulumi Go code is here  
    })  
}
```

The easiest way to understand this difference is to consider the way each program is invoked. A Pulumi Automation API program written in Go is executed by running `go run main.go`, like running any other Go program. In contrast, a Pulumi Go program is invoked by the `pulumi` CLI.

Zephyr's Automation API program starts out with checking for a command-line argument to indicate the desired mode of the operation—should it be creating/ updating resources, or destroying resources? In a standard Pulumi Go program, this would be determined by the invocation of the `pulumi` CLI (for example, whether you ran `pulumi up` or `pulumi destroy`). Here, though, that falls to the user to check:

```
// Determine mode of operation; default is to refresh/update
destroy := false
argsWithoutProg := os.Args[1:]
if len(argsWithoutProg) > 0 {
    if argsWithoutProg[0] == "destroy" {
        destroy = true
    }
}
```

Next, the program reads some configuration data from a YAML configuration file; this configuration data includes things like the relative file system location of the Pulumi programs that this Automation API program is going to execute:

```
// Get configuration data
data, err := os.ReadFile("config.yaml")
if err != nil {
    fmt.Printf("Error reading configuration file: %v\n", err)
    os.Exit(1)
}
var env Environment
err = yaml.Unmarshal(data, &env)
if err != nil {
    fmt.Printf("Error parsing configuration information: %v\n", err)
    os.Exit(1)
}
```

Separating configuration from code in this manner allows you to use the language's existing ability to create binary executables that can be easily distributed to your teams. For example, Zephyr has the option of running `go build` to create a binary executable for distribution to internal teams.

From there, the Automation API program gets into [setting up the stacks](#) it will use:

```
// Set up base stack
baseStackName := auto.FullyQualifiedStackName(env.Organization, env.BaseProject.Name,
env.StackName)
baseStack, err := auto.UpsertStackLocalSource(ctx, baseStackName,
env.BaseProject.Location)
if err != nil {
    fmt.Printf("Failed to create or select %s stack: %v\n", env.BaseProject.Nickname, err)
    os.Exit(1)
}
fmt.Printf("Successfully created/selected %s stack\n", env.BaseProject.Nickname)
```

In this particular program, Zephyr decided to start with a local workspace. (You can read a bit more about workspaces in the context of Pulumi Automation API [in the docs](#).) This is reflected in the use of the `UpsertStackLocalSource` method shown above, which will create or select a stack for a Pulumi program found in a local filesystem location. Users can also use `UpsertStackInlineSource`, which embeds the Pulumi program into the

Automation API program as a function. There are trade-offs to each approach:

- Using local source (i.e., the Pulumi program is stored outside of the Automation API program) has the benefit of allowing users to independently revise the Pulumi program or the Automation API program; this may be the ideal approach where organizations have separate teams that will create, manage, or maintain the Automation API programs. In Zephyr's case, where there is a platform team, this made a lot of sense for them. This does also mean, as you might guess, that the Pulumi programs must be reachable by the Automation API program (this typically means users need to clone down the appropriate repositories to their system first).
- Using inline source (where the Pulumi program is embedded in the Automation API program) may be a better fit when there is a single team managing or maintaining both the Pulumi program(s) as well as the Automation API program(s), or when—for whatever reason—these programs should not be revised independently of one another. This has the benefit of removing the need for users to clone or copy separate Pulumi programs onto their system before being able to use the Automation API program.

After all four stacks—base, platform, data, and app—have been set up for use within the Pulumi Automation API program, then comes the “real work”: performing operations on those stacks. To reduce duplicate code, the Zephyr program makes use of three functions:

1. The `refreshStack` function (starting on [line 199](#) of `main.go`) performs the equivalent of a `pulumi refresh` against a stack.
2. The `updateStack` function (starting on [line 218](#) of `main.go`) performs an update against the stack. This is like running `pulumi up` against a stack.
3. Finally, the `deleteStack` function (starting on [line 236](#) of `main.go`) destroys the resources in a stack. This equates to running `pulumi destroy` against a stack.

Using these functions, [lines 94 through 197](#) perform the key task of taking action to stand up or tear down the Zephyr online store and all associated infrastructure. If the user passes the parameter “destroy” on the command line (meaning the Pulumi Automation API program is invoked using `go run main.go destroy`), then it will tear down the stacks. Otherwise, the Pulumi Automation API program will perform a refresh and then an update—much the same way that the `pulumi CLI` itself behaves.

Along the way, you can see the Pulumi Automation API program passing configuration values to the stacks being orchestrated (this is [lines 177 through 182](#), for example):

```
// Set config values for app stack
appStack.SetConfig(ctx, "platformOrgName", auto.ConfigValue{Value: env.Organization,
Secret: false})
appStack.SetConfig(ctx, "dataOrgName", auto.ConfigValue{Value: env.Organization, Secret:
false})
appStack.SetConfig(ctx, "platformProjName", auto.ConfigValue{Value:
env.PlatformProject.Name, Secret: false})
appStack.SetConfig(ctx, "dataProjName", auto.ConfigValue{Value: env.DataProject.Name,
Secret: false})
appStack.SetConfig(ctx, "platformStackName", auto.ConfigValue{Value: env.StackName,
Secret: false})
appStack.SetConfig(ctx, "dataStackName", auto.ConfigValue{Value: env.StackName, Secret:
false})
```

Among other things, this means you can take your existing Pulumi programs, and—without making any changes—orchestrate them using an Automation API program! Further, in reviewing the code shown above, you can see that the Pulumi Automation API ties into [Pulumi’s secrets functionality](#).

In the end, using Pulumi's Automation API enabled Zephyr's platform team to simplify the workflow for standing up the online store application and all of its associated infrastructure into a simple and straightforward process:

1. Developers ensure they have cloned the appropriate source repositories onto their system.
2. Developers run `go run main.go` (or `go run main.go destroy`) to stand up or tear down the online store and all associated infrastructure.

Although Zephyr uses multiple GitHub repositories, the Pulumi Automation API has no requirements for multiple repositories. You could use a monorepo, if that better suits your particular requirements.

Summarizing Topic #6

This section discussed the following recommended practices when using Pulumi Automation API:

- Use the Automation API to orchestrate operations on multiple stacks. In cases where you've split Pulumi projects, you've introduced a potential opportunity for using Automation API to orchestrate operations. This is a great counter-balance to offset the relative increase in complexity that comes from breaking large Pulumi projects into smaller, more focused Pulumi projects.
- Separate configuration from code. This is already a generally-accepted recommendation in most programming circles, and it applies to Automation API programs as well. Doing so enables you, if you wish, to distribute binary artifacts to teams instead of requiring them to execute the source code of your Automation API programs.
- Use local source to decouple Automation API programs from Pulumi programs when separate teams are involved. When separate teams are creating, managing, or maintaining the Automation API programs and the Pulumi programs, using local source instead of inline source enables the teams to operate somewhat independently.
- When there is a single team managing both Automation API programs and Pulumi programs, use inline source. This is also true when, perhaps due to compliance or regulatory concerns, the Automation API program should be more tightly coupled to the Pulumi programs it is automating.

Conclusion

Future Considerations

We've stated before that recommended practices (or even "best practices," if you prefer that term) are always point-in-time recommendations based on the requirements of the organization/team/individual using Pulumi. What this means, in practice, is that there is almost always room for growth and evolution. Organizations, teams, and individuals rarely remain static, and therefore their use of Pulumi is also unlikely to remain entirely static.

So, what are some potential areas of future growth for Zephyr?

- Splitting the deployment of the microservices: Right now, all the microservices that comprise the online store are deployed together. What if one particular team needs to deploy only their service? The current approach can certainly accommodate that—Pulumi is declarative and will only change what needs to be changed—but what's better for the Zephyr team? Does Zephyr need to optimize for allowing service teams to move independently? Is it about reducing blast radius, so that undesired changes aren't introduced to another service? These are the sorts of considerations that drive this decision.
- Using component resources: Currently, the Zephyr team is using the standard API objects supplied by the Pulumi provider. For example, the various microservices for the Zephyr online store are defined using standard Kubernetes objects, like this:

```
const assetsNs = new k8s.core.v1.Namespace("assets-ns", {
  metadata: ...})

const assetsService = new k8s.core.v1.Service("assets-service", {
  metadata: ...})

const assetsDeployment = new k8s.apps.v1.Deployment("assets-deployment", {
  metadata: ...})
```

- Component resources would allow the Zephyr team to define a single logical resource that contains “child” resources. This would allow the Zephyr team to create a Microservice resource that incorporates the standard Kubernetes Namespace, Deployment, and Service objects. Deploying an instance of a Microservice resource deploys all the child objects defined in the component resource. This can simplify the Pulumi code through further use of DRY (Don’t Repeat Yourself).
- Making use of Pulumi Deployments: Zephyr’s CI/CD pipelines are not a topic we’ve discussed in the walkthrough. Given that Zephyr uses GitHub, they could use GitHub Actions (GHA); Pulumi has an action for GHA workflows. Zephyr could also use Pulumi Deployments, which offers an extensive set of features—OIDC support for dynamic credentials, Review Stacks (per-PR stacks), support for GitHub Enterprise, and GitOps-style workflows, to name a few. Support for Pulumi Deployments is also baked into Automation API, meaning that Zephyr could extend their existing Automation API code to incorporate Pulumi Deployments.
- Addressing secrets and configuration management: We’ve also not discussed how Zephyr could manage their secrets and configuration data. Pulumi ESC is a good option here, offering seamless integration with Pulumi for infrastructure as code while also supporting a variety of other DevOps tools and use cases (here’s one example).

Even though it’s working well, clearly there’s a lot of room for the Zephyr team to continue to enhance and improve their infrastructure as code implementation!

Key takeaways

Throughout the guide, we’ve provided recommendations on how to best utilize Pulumi. While we recommend reading the entire guidebook, this section will “summarize our summaries” and provide some broad, overarching recommendations.

1. With regard to Pulumi projects, keep it as simple as possible, but no simpler. While tools like Automation API can help with orchestrating multiple projects, it remains true that in most cases a single project is simpler. Use multiple projects when necessary, but only when necessary.
2. Use stacks freely. Using multiple projects has a cost in complexity. Using multiple stacks, on the other hand, does not—so use them freely to model different environments, to provide per-developer test environments, to test changes before rolling them into production, or for any other use case where you need a separate, independent instance of the infrastructure defined in your Pulumi program.

3. Focus on the requirements. It can sometimes be too easy to focus on the technology instead of the requirements. There are a lot of very cool things you can do with Pulumi, but try not to get distracted from the reason you're using infrastructure as code: repeatable, reliable, consistent deployments of your cloud infrastructure.
4. Don't forget about security along the way. Make sure you are properly securing your infrastructure. Pulumi [CrossGuard](#) can help here by providing "policy as code" guardrails for cloud infrastructure. Pulumi Cloud's RBAC functionality helps with ensuring the right users have the right permissions to the right sets of cloud infrastructure resources.

Reviewing the code

You can view the code we've presented in this guide in the following GitHub repositories:

- The ["zephyr-infra" repository](#) contains the base infrastructure definitions.
- The ["zephyr-k8s" repository](#) contains the Pulumi code to stand up the Kubernetes cluster onto which Zephyr deploys their online store application.
- The ["zephyr-data" repository](#) manages Zephyr's "data layer" for persistent data used by the online store application.
- The ["zephyr-app" repository](#) is Zephyr's online store. This repository has both the application code as well as the Pulumi code for deploying the application (see the "infra" directory).
- In the ["zephyr-automation" repository](#) you'll find Automation API programs used by Zephyr to orchestrate deployments of the different projects involved in supporting the online store.

In each of these repositories, there are different branches that reflect the state of the code as of a particular section. The "main" branch reflects the final state of the code, as described in this wrap-up section.



Throughout this guidebook, we've explored key principles and practices for effective infrastructure as code at scale, illustrating them through Zephyr's platform engineering journey. While their specific implementation used Pulumi, the core concepts—modularity, self-service, security, compliance, and automation—are universally applicable. The primary goal of platform engineering is to empower developers with self-service capabilities, enabling faster innovation and streamlined workflows, while ensuring security, governance, and compliance remain intact.

Pulumi's unique programmable infrastructure approach enables teams to apply these practices in powerful ways. However, the patterns demonstrated provide a valuable framework for any organization building modern infrastructure platforms, regardless of tooling.

As you evolve your own infrastructure practices, consider how these principles can help you deliver secure, reliable platforms that enable developer productivity. Experiment with the patterns that align with your needs to accelerate your platform engineering journey. Take the next step by [registering for a Platform Engineering Immersion Day](#) or [speaking to one of our representatives](#).



pulumi.com



github.com/pulumi



[@pulemicorp](https://twitter.com/pulemicorp)